

Programming a Robotic Toy with a Block Coding Application: A Usability Study with Non-programmer Adults

Muhammet Ramoğlu^{1,2(✉)}, Çağlar Genç³, and Kerem Rızvanoğlu⁴

¹ Graduate School of Science, Engineering and Technology,
Industrial Product Design Department, Istanbul Technical University, Istanbul, Turkey

² Arçelik A.Ş., Industrial Design Directorate, Istanbul, Turkey
ramoglu@itu.edu.tr

³ Koç University – Arçelik Research Center for Creative Industries (KUAR), Istanbul, Turkey
cgenc14@ku.edu.tr

⁴ Faculty of Communication, Galatasaray University, Istanbul, Turkey
krizvanoglu@gsu.edu.tr

Abstract. Recently, sophisticated robotic toys have commercially emerged into our lives. Apart from being only a toy, some of these smart devices are programmable for accomplishing commands given by the end-user. However, usually, end-users are not experts in robotics or programming. In order to explore the usability issues related to the non-programmers' experience of controlling the robotic toys, we conducted a user study with non-programmers (N = 9) by using *Sphero* (a robotic toy) and tested its mobile application, called *SPRK Lightning Lab for Sphero*, which adopted visual programming language with a block-based coding interface. Our procedure consisted of a pre-test and a semi-structured post-test interview as well as an exploring session for the participants and three tasks with a short semi-structured interview at the end of each task. Our findings, which highlighted the usability issues of *SPRK Lightning Lab for Sphero* application, contribute to the field by providing design suggestions on using a digital medium and a tangible device together, the usability issues of block coding by non-programmers and learnability in a robotic toy application.

Keywords: Robotic toys · Visual programming · Block coding · Usability test · Mobile application

1 Introduction

Recently, different types of robots have emerged into our lives such as drones and service robots. Among them, also robotic toys that are capable of jumping, rolling, changing colors and moving autonomously have recently become popular for both children and adults. They differ from traditional similar toys such as remote control cars by being programmable for

This study was realized through the coordination of Assoc. Prof. Kerem Rızvanoğlu with the support of Galatasaray University Scientific Research Fund (Project ID: 16.300.006).

autonomous actions and by integrating with a mobile device to play mixed reality games. For example, Lego Mindstorms¹, Ozobot², and Dash & Dot³ have been commercialized as educational robot kits to teach coding to children by providing digital interfaces for programming these toys. On the other hand, robotic toys such as Sphero and Ollie⁴ have been also used by non-programmer adults for entertainment and Lego Mindstorms used in universities to teach programming [1]. Non-programmer adults are also using similar robotic toys for entertainment. Thus, the usability issues related to their experience of controlling robotic toys are critical for designing these interfaces.



Fig. 1. Sphero, a robotic toy

User experiences of several visual programming languages have been examined in previous studies to program robots. Among them, *block coding* metaphor has frequently used where graphical code blocks are used instead of text-based programming syntax for creating commands. Researchers found that users found code blocking easier to learn than text-based programming [2–5]. Besides, researchers evaluated the user experience of a block coding applications with children [2, 6, 7] and highlighted the effect of using block coding to program robotic toys, on developing children's certain skills [7–9]. However, to our knowledge, there is no study on the usability of mobile block coding application to program a robotic toy with adults. This kind of understanding is important due to the existence of commercialized robots in our daily lives. We believe that the use of block coding

¹ <https://www.lego.com/en-us/mindstorms>.

² <http://ozobot.com/>.

³ <https://www.makewonder.com/>.

⁴ <http://www.sphero.com/>.

and similar visual programming concepts will increase in the future for programming robotic toys as well as other interactive devices such as home appliances. Therefore, knowing usability issues in such novel systems will be crucial to enhance the user experience of these devices and applications in the future.

The aim of this study is to examine (1) usability issues when digital (mobile application) and physical (robotic toy) media are used together, (2) user experience of non-programmers while programming a robotic toy with block coding and (3) learnability in a mobile robot programming application. For this purpose, we conducted usability tests of the application called *SPRK Lightning Lab for Sphero* which uses block-based coding interface and controls the robotic toy called *Sphero* (see Fig. 1).

2 Background

2.1 Block Coding

Visual programming languages have wide application areas such as 3D modeling with Grasshopper 3D⁵, video game making with Blender Game Engine⁶, and music generation with AudioMulch⁷. Besides these, visual programming has been popularly used for teaching coding to non-programmers, especially to children. The basic principle of block coding is drag and drop existing code blocks and snap blocks together to generate a program or series of commands. Code blocks are easy to relate with other blocks with colors and shapes. LogoBlocks [10] was one of the earliest examples of using blocks in visual programming and introduced programming to many students [11]. Begel [10] claimed the advantages of visual programming such as using real life metaphors for understanding the functionality better, its easy browsability (easily noticing the structure of a program comparing to textual program) and its potential of representing the relation among code blocks. After LogoBlocks, block coding metaphor has been applied in various software and visual programming languages such as Scratch [12], BrickLayer [13], Blockly⁸ and Snap! [4]. Today, MIT's Scratch and similar block coding languages are being used by millions of people, and not only elementary schools but also high ranking universities have been using block coding to introduce programming [11]. Moreover, with the increase in ubiquitous computing and smartphones, block coding has also become available in mobile devices and tablets. For example, Scriptkit, Lightbot and Kodable are mobile applications that focus on teaching programming to children with block coding [14].

2.2 Robotic Toys

Block programming is mostly used for teaching programming or making a particular system programmable to non-programmers. A robotic toy is an example of both a

⁵ <http://www.grasshopper3d.com/>.

⁶ <https://www.blender.org/>.

⁷ <http://www.audiomulch.com/>.

⁸ <https://developers.google.com/blockly/>.

programmable system and an educational tool that uses block coding and aims at non-programmers. Elkin et al. [7] stated that robotic toys provide a playful way to learn to program and it improves certain skills of children such as problem solving, planning [8], and even social skills [9]. Lego Mindstorms NXT was one of the first commercial programmable robotic toys with block coding and released in 2006 [15]. Klassner and Anderson also claimed that Lego Mindstorms is an affordable alternative for teaching programming and practice robots in college level [1]. Robotic toys become popular in the last decade and various robotic kits have been released such as Kibo, Dot & Dash, Ozobot, Cubelet and mBot. These robotic toys come with mobile applications to control and program robots for autonomous actions. In addition, mobile applications such as Tickle⁹ and Tynker¹⁰ are designed to program different robotic toys in one block coding application.

Robots such as Ollie and Sphero by Sphero Inc. (previously Orbotix) are both aimed for adults and children with entertainment and programming education. Ng, Chow and de Lima Salgado [16] indicated that robotic toys such as Sphero bring novel content in mobile applications with different controlling methods and sensors. Sphero has various applications to change its colors with music, play with your pet, and even play mixed reality games. In mixed reality games, users see their robot through a virtual environment from the screen of their smartphones or tablets with the help of a camera. For example, *Sharky the Beaver* application shows Sphero as a beaver on the screen of a tablet and provides to control the beaver in your physical space. Robotic toys have been targeting adults as well as children and these robots brought new content to mobile applications with usability issues.

2.3 Usability of Block Coding

Block coding applications and robotic toys have been evaluated by users in previous studies. Dill et al. [5] found that students improve their coding skills by programming movements of a robot in a prototype game with block coding. Elkin et al. showed that children between 3 to 5 years old were able to create simple programs with block coding [7]. However, children had problems with the task that include *repeat loop*. Loop function in programming is used to repeat the code until a condition happens. For example, to change the robot's color when it is grabbed, you need to loop the code otherwise, the system will read the code, check if it is grabbed and if it is false, it will exit the code without changing anything. To use loop code, users need to know that the computer runs the code lines rather fast, runs a code line once and stops running at the end of the code if not repeated in a loop. Ramirez-Benavides, Lopez, and Guerrero [2] evaluated the usability of TITIBOTS, a mobile application for robot programming, in terms of learnability, efficiency, memorability, errors and satisfaction. The usability test was conducted with children aged 4 to 6 years old who are unable to read. They indicated that all children like the system but when they ask to draw children what they liked, 62% drew a robot, 31% drew a tablet and rest drew both of them together. Therefore, it is

⁹ <https://tickleapp.com/>.

¹⁰ <https://www.tynker.com/>.

important to separate a robotic toy by mobile application in an evaluation of a usability test as users might comment on the attractive robot instead of the application. Researchers also pointed out that it was hard to evaluate the effect of teaching programming concept by using block coding on learning programming logic in the long term. Pane [6] asked children to write a statement after showing a logic to move the Pacman and analyzed their approach. For example, 67% of children used images in their statement that show they felt comfortable to express a statement with visuals. In addition, 58% of participants used *when*, *if* and *after* and more than 60% used *and* and *or* in their statements similarly to programming syntax. This study showed that non-programmers' approaches were partially similar to programming logic but, non-programmers did not always express a statement by the existing programming logic. Thus, block coding might be conceptualized without only mimicking the existing programming syntax.

Ketola and Roto [17] noted that learnability is an important element in user experience and exists in different usability measures [18–20]. Hung [21] analyzed the learnability of several block coding platforms such as Scratch, App Inventor, Stencyl and GameBox. Hung found that video tutorials were common in these block coding systems. GameBlox included a help page on the side of the screen and Scratch provided a help button that pops up a help page on the side of the screen. Scratch also included a step-by-step tutorial that teaches user by practice. However, learnability of these block coding platforms was not evaluated in this study with a user test.

Researchers showed that block coding is easy to use, and it is a useful tool for teaching programming with the help of robotic toys. However, previous studies did not examine user experience of a block coding application to control a mobile robot with adults. Block coding applications and mobile applications of robotic toys have been increasing. We believe, block coding will be widely used to program numerous systems by non-programmers in the future. Thus, we conducted a study with 9 adult users to examine usability issues in SPRK Lightning Lab for Sphero application that programs the robotic toy Sphero.

3 SPRK Lightning Lab for Sphero

The Sphero has multiple controllable sensors, motors, and LEDs which pave the way for the end-user to program its functions. SPRK is one the applications for controlling this robotic toy. Different from other applications with the same aim, SPRK has a visual programming interface, which has pre-defined code blocks. These code blocks are presented in an interface (see Fig. 2) where a user can drag and drop them in order to create a series of commands, which can be saved or sent to the Sphero to be executed. Users can experiment by adding, removing code blocks or mixing the order of code blocks. The code blocks vary from Sphero-specific pre-defined blocks like actions (i.e., rolling, setting the heading, spinning and setting color), sensors (i.e., heading, speed, accelerometer) or events (i.e., on collision, on free fall) to more textual programming tools as in format of code blocks like delay, loop or if/then. Most of the Sphero-specific code blocks have parameters that the user can edit like in the case of *roll* block. In *roll* block, there are parameters of rolling duration, heading angle and the speed of the

Sphero. Also, some of the code blocks can be dragged in another code block for different purposes.

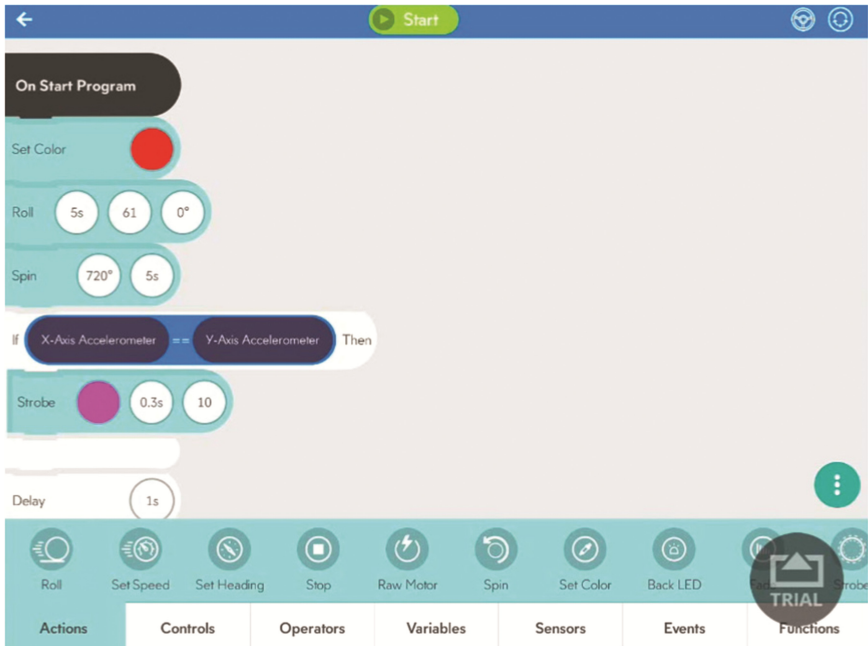


Fig. 2. Overview of SPRK Lightning Lab for Sphero application. The screenshot also included an example usage of *if/then* block. Sensor blocks of *accelerometers* were placed on the first row of *if/then* block where the condition was defined and the code block called *strobe* was placed in the *if/then* block to be executed if the condition was true.

For example, as shown in Fig. 2, *if/then* command has the first line of the block for defining a condition in which sensors can be dragged in to define a parameter for describing a condition. *Operators* can also be used to define the relationship among variables on condition. Additionally, for *if/then* block, the user can drag actions into the code block to be executed if the condition is true. Finally, in this application, users can also create their own actions or variables. When the user touches the *start* button on the top, Sphero starts executing the program in the real world.

The SPRK application has a tutorial which includes a walkthrough, starting with guiding user to arrange the orientation of Sphero, continuing with showing the basic features of programming with blocks (i.e., double tapping for getting information about a code block on the bottom bar, dragging a new code block, deleting a code block) and executing the program. That tutorial uses hint bubbles for guiding the user through these basic features. The application also has a section where some ready-to-use programs are present that the user can investigate to learn how the code blocks can be used.

4 Method

This research aimed to test non-programmers' user experience while controlling robotic toys. Therefore, we focused on one of the most famous commercial robotic toys called Sphero and one of its applications (SPRK Lightning Lab for Sphero) to highlight usability issues. We included three tasks for the participants and semi-structured interviews to accomplish at the beginning and at the end of the sections. We also asked open-ended questions after each task. We mainly collected qualitative data that was supported by quantitative data. The rest of the section will describe the details of our method.

4.1 Participants

Nielsen and Landauer recommended that 5 participants are enough to detect usability problems from a qualitative usability study [22, 23]. We conducted our study with 9 participants (6 females) who had never used Sphero or any of its applications before. The study was conducted with Turkish speaking participants to eliminate any cultural factors. They were 3 graduate and 6 undergraduate students from Koç University who volunteered to our announcement that was on social media. Participants' ages ranged from 18 to 32 ($M = 23.00$, $SD = 3.70$) They did not have any programming experience or had little knowledge about programming. None of them were familiar with visual programming languages. All of them had experiences of using a tablet.

4.2 Apparatus

The Sphero 2.0, a robotic toy with a spherical shape was used in our study. Sphero has features such as moving on the ground by rolling, changing colors and, sensing its own movement and direction with the help of the accelerometer and gyroscope. SPRK Lightning Lab for Sphero (version 1.2.0), an application specifically designed for the Sphero, was tested by using an Apple iPad (iOS 9.2) during the study. This application is based on a visual programming language, and contains codes that are embedded in blocks. By arranging code blocks, a user can program the Sphero for controlling its actions. All processes were recorded by a webcam and AirServer¹¹ software was used to mirror the screen of iPad to a laptop computer. We also recorded the mirrored screen by screen recorder software to match the gestures and the voice of the participant along with their actions taken on the application. The task completion times were also documented.

4.3 Procedure

The study took place in a university classroom with an empty surface to move Sphero. The participants were sitting on a chair. Before each test, we introduced ourselves, declared our aim as exploring their experience with the application while controlling

¹¹ <https://itunes.apple.com/tr/app/airserver-connect/id967004087?mt=8>.

the Sphero. We explained to them that we were not testing them; instead, we were testing the application. We asked them to follow think-aloud protocol [24] in which participants were supposed to tell simultaneously what they were thinking while using the application. Then, we informed them about the schedule of the test. Additionally, we told them to ask if they need translation for the words on the application. Overall, the tests took approximately 45 min.

The rest of the sub-section will inform the readers about the details of the procedure in the actual sequence of the study.

Pre-test Interview. During pre-test interview, we asked participants their demographic information (i.e., age, academic degree) and how they rate their experience on tablet usage and programming from zero to expert. Additionally, we asked them if they had any knowledge of block-based programming. In exploration session, we gave the tablet and introduced the Sphero to the participants. After opening the application, the logic of block coding was explained. We guided them to begin with the tutorial in the application. We also pointed out that there is a *Sample Codes* section that they might take a look. The time was limited to 5 min for this session. After 5 min, we asked the participants to rate how exploration session assisted them to understand the application with 7-point Likert scale (1 - Not at all instructional, 7 - Extremely instructional). Based on their response we encourage them to reason their response with open-ended questions.

Tasks. Three tasks were given to participants to accomplish in these sessions. The first and second tasks needed participant to use basic functions of the Sphero such as rolling and changing its color. The second task was more complex than the first task. The aim of the first and second tasks was to analyze how they experience overall interface (i.e., dragging/removing a code block, editing parameters of those blocks). The third task was more focused on programming logic where the user needed to use *if/then* and *loop* code blocks. We wanted to investigate the experience of participants when the code block was more complicated. For example, to use *if/then* block, a user needs to drag other code blocks into *if/then* block parenthesis.

In the beginning of all tasks, the Sphero was placed in front of the participant's chair. We reminded participants that they could look at *tutorial* or *sample codes* sections any time they needed. We included time limits for all tasks but extended it if the participant was close to accomplishing the task since our aim was to discover usability issues. The first task was to make the Sphero go approximately 2 m away from the participant. The time limit for this task was 5 min. In the second task, again, we asked participants to move the Sphero app. 2 m away from them. But this time we specified that the color of the Sphero should be red in the beginning. After going app. 2 m, it should wait for a while. Then, the color should turn to green. Finally, it should come back. The time limit was also 5 min for this task. The third task was defined as blinking the Sphero while it was spinning for the first 4 participants. Then, we realized that the task was too hard for participants. Thus, we replaced it with another task that participants would use similar functions. The third task included changing the color of Sphero if it was thrown up in the air. We specified that the Sphero should be green at the beginning and asked them

to make the color of the Sphero blue in case of being thrown. Different than the other tasks, we asked participants to use specific code blocks: *if/then* and *loop* blocks. We explained that *if/then* command refers to a condition in programming languages. We told them that this command makes defined commands work only if the defined condition occurs. Additionally, we mention that this application reads a block code in nano-seconds and passes to the other one. *Loop* command was explained to overcome this situation by making the application read the code blocks inside it repeatedly. We included a time limit of 10 min for this task. Our aim was to explore how non-programmers use commands like *if/then* and *loop* and how they interact with the user interface of these features. At the end of both tasks, we asked participants to rate the difficulty of the task by a 7-point Likert scale (1 - Not at all difficult, 7 - Extremely difficult). After their response, we asked open-ended questions to justify their quantitative response. We also asked them what they liked and disliked when they were using the application after each task.

Post-test Interview. The beginning of the semi-structured post-test interview involved open-ended questions such as “What would you change in the application?” and “What was the most difficult thing for you? Why?”. After open-ended questions, we asked 11 questions that were responded in 7-point Likert scale. The questions (Table 1) contained several subjects related to the usability of the application such as the evaluation of tutorial as the evaluation of tutorial, the ease of use, the pleasure of use, self-confidence while using the application. Questions 2, 3, 4, 5, 6 and 10 were based on System Usability Scale (SUS) [25] with small modifications. We asked respondents to explain the reason behind the given rating to collect both qualitative and quantitative data.

Table 1. Questions in the post-test interview

Q1 - Tutorial was helpful in using the application
Q2 - I found the system unnecessarily complex
Q3 - I thought the system was easy to use
Q4 - I think that I would need the support of a technical person to be able to use this application
Q5 - I would imagine that non-programmers would learn to use this system very quickly
Q6 - I felt very confident using the application
Q7 - I found the given tasks difficult
Q8 - I think I learned the application and I feel confident to use it
Q9 - I enjoyed using the application
Q10 - I think that I would like to use this application again
Q11 - I would recommend the application to my friends

5 Results and Discussion

5.1 Quantitative Results

In the first tests, we asked 4 participants to flick Sphero when it is turning for task 3. When we realized that this task is too complex we changed it to the task explained in

the procedure section. Because our focus was on the usability of the application and the mentioned change in Task 3 did not affect the scope of the task, it did not make a major impact in the results. Average difficulty of task 3 was 6.11 with a standard deviation of 0.78. When we exclude the first 4 users from the analysis, the mean value of the results increased to 6.20 ($SD = 0.84$). Mean values of the task difficulties (see Fig. 3) suggest that task 2 was the easiest. The reason for that might be that the participants got used to the user interface in the first task and their knowledge from the first task help them to accomplish the second one easily. The hardest task for the participants was the third task. This might be due to confusing nature of the task in terms of understanding different code blocks which were directly related to the textual programming (i.e. if/then and loop). Another explanation would be that the user interface was too abstract for them to understand the functions of those code blocks, thus it was hard for them to construct a program by using *if/then* and *loop*. Elkin et al. [7] found similar results with children, most of whom failed in the task with the *loop* function. We discussed these issues with details in the qualitative analysis section.

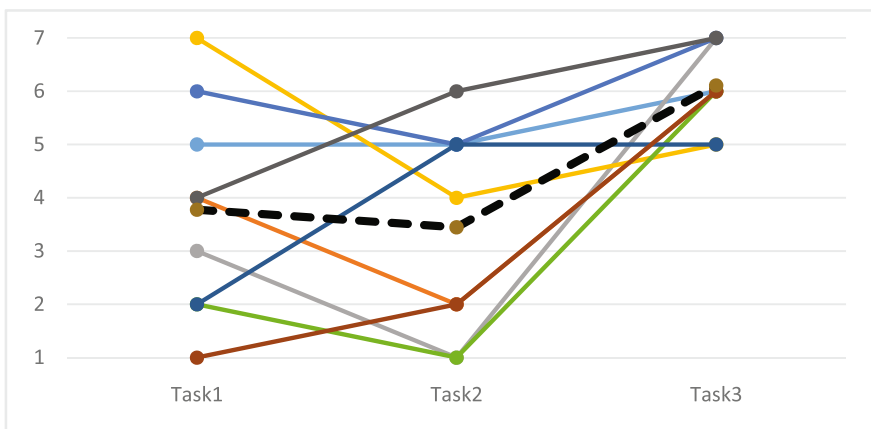


Fig. 3. Participants were asked to assess the difficulty of tasks after each task. Their response is shown in 7-point Likert scale and dashed line shows the average.

We can consider answers of Q3 and Q4 as a neutral in the post-test questionnaire. When we examined the extremes points in the graph (see Fig. 4), we noticed that participants agreed to the Q9, Q10, and Q11 in the post-test questionnaire, which shows the application was engaging for the participants. However, some of the participants tended to comment on the robot instead of the application when they were explaining what they liked about the system. Ramirez-Benavides et al. [2] also found that children expressed that they were interested in robot more than in the application. Q2 and Q7 had been rated with low scores that indicated participants did not think that using the application was hard for them. Besides, high scores of Q5 and Q6 supported the inference that participants used the application without having difficulties.

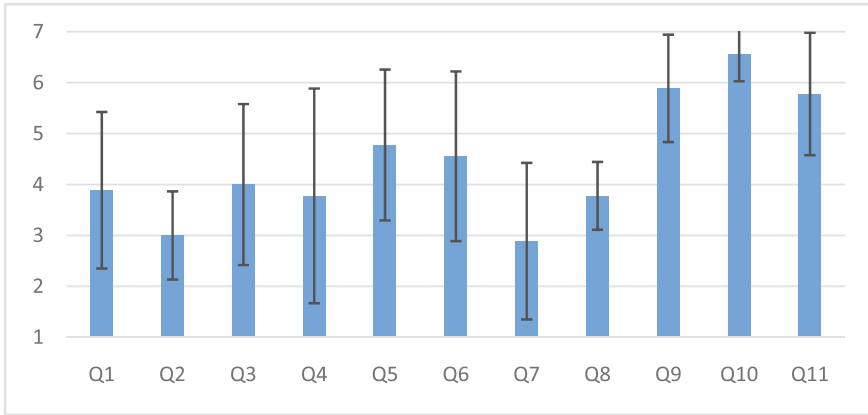


Fig. 4. Mean values and standard deviation of the agreement of the participants with the phrases in the post-test questionnaire.

The result of comparison between the evaluation of tutorial just after the exploration session and the evaluation of post-test questionnaire showed that participants' perception of the helpfulness of the tutorial has been affected negatively after accomplishing the tasks. The reason for this might be that the tutorial does not cover the essential information that might help the participant to complete the given tasks.

5.2 Qualitative Analysis: Observations and Questionnaires

Tutorial. The scope of the tutorial was criticized by the participants, especially in the post-test interview which can also be seen on the graph in Fig. 5. They found it shallow and supported the idea that a more detailed tutorial should be implemented in order to make it more useful for learning the application. Participant 7 suggested the idea of a more detailed walk-through tutorial where participants were guided for building one of the sample programs that were present in the *sample programs* section. Participant 1 mentioned the possible benefits of walk-through videos shown as a tutorial at the beginning. She claimed that this might be a good way of introducing more complex parts of the application. Furthermore, the hint bubbles on the tutorial failed to give intended instructions about the basics of the graphical user interface of the application. Although we encouraged the participant to follow the tasks shown on the hint bubbles in the tutorial, most of the participants did not accomplish all the hints shown in the tutorial. Moreover, some of them did not recognize that there were hint bubbles in the first place. One of the participants, while exploring the tutorial again during the post-test interview, said: “Now, I noticed that I needed to follow the instruction on the tutorial!” (P5). When we asked “What do you want to change in the application?” in the post-test interview, P1, P7, and P9 mentioned that they want to change tutorial.

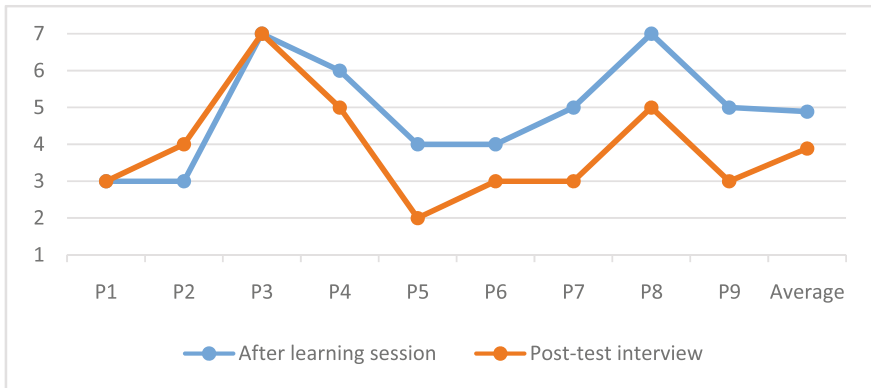


Fig. 5. Evaluation of the tutorial in terms of its usefulness in learning the application after exploration session and in the post-questionnaire.

Block-Coding. Dragging and dropping code blocks were engaging and easy to use for the participants. Most of the terminology and the icons assigned to the code blocks were understood by the participants. Some users mentioned that the different colors used for a different group of code blocks were well designed. Almost all the participants immediately understood the logic of block coding (creating a flow of actions for the Sphero to execute). However, there were some critical usability problems on this subject. Some users had troubles with terminologies of textual programming languages (i.e. *if/then*, *delay*). Additionally, the parameters of some block codes were irrelevant for most of the participants and they had troubles understanding them. For example, to move the Sphero, the participants used the *roll* command (i.e. first and second task), they were searching for an input area for entering the distance, instead of entering the duration, speed, and heading angle. Moreover, the parameters of code blocks like *roll* did not have any units, for example, meters or meters/second, and they were not understood. Participants also mentioned that when they put same code block for the second time, the application should automatically set the same values for the parameters as the first code block.

Almost all the participants had troubles with the *if/then* code block. As we described before in the paper, an *if/then* code block needs a condition and defined actions. Conditions were supposed to be defined in the first line of the code block. When a participant added an *if/then* code block, the first line had “0 == 0” which did not mean anything to most of our participants since they were non-programmers. Although they could add other code blocks such as *accelerometer* into condition line of *if/then* block to define an accelerometer condition. Besides, the graphical clues on the *if/then* code block failed to give clues about which code blocks could be dragged on this space.

The Relation of the Digital and the Physical Environment. The most challenging part of the first two task was to orient the Sphero in the real environment. Almost all the participants, at some point of at least one task, moved the Sphero in a wrong direction. Although some of them noticed or remembered that they needed to arrange the aim of the Sphero, the others had problems in figuring out the Sphero’s orientation in the real

environment. One participant even tried to change the orientation of the Sphero by holding and rotating it by hand, instead of opening the orientation interface by touching the icon on the top-right. This might be due to the fact that the icon assigned to modifying the orientation was small and abstract. Two of the participants, P3 and P7 mentioned that the small size of the aiming icon should be increased. Another issue about the uneasy relationship between the programmed actions in the interface and the physical world revealed in the requests and comments of participants which were: “I have the tablet; I could use some of its properties. I could define the place with the tablet’s camera. It would be nice if I could draw (its movements) on the tablet.” (P4), “It (moving the robot) could have been made simpler without an accelerometer and if/then, such as drawing it” (P6).

Overall, the responses of the participants on post-test interview suggest that visual programming languages based on block-coding create an engaging experience and an easy-to-learn platform for controlling robotic toys. However, our observations and semi-structured interviews suggested that designer of those interfaces should consider some issues about the usability such as;

- Supportive contents (i.e. tutorials) are critical for non-programmers to learn how to use those devices. Thus, tutorials should contain more details about complex aspects of the applications such as *if/then*. Besides initial tutorial, users should be able to access explanatory information about features while programming.
- Terminologies and metaphors used in those kinds of applications for controlling robotic toys should consider the mental model of non-programmer users and should be simplified and familiarized for the end-users.
- The visual features of the blocks should support the functional relation between code blocks. For example, if some code blocks can be dragged into another one, there must be visual clues that reflect the relationship.
- The relation between the digital user interface and the real environment (including the robotic toy and surrounding environment) should be considered while designing the user interface to create an understanding of the spatial relationship between them. More specifically, for the robotic toys that move in the space, the orientation information should be specified to decrease the errors.

6 Conclusion

Our study aimed to reveal the usability issue of visual programming interfaces by block-coding for controlling commercial robotics toys. In order to explore the usability issues related to the non-programmers’ experience of controlling the robotic toys, we conducted a user study with non-programmers (N = 9) by using Sphero (A robotic toy) and tested its mobile application, called SPRK Lightning Lab for Sphero, which adopted visual programming language with a block-based coding interface. We included an exploring session, three tasks, pre-test, and post-test interviews as well as semi-structured interviews at the end of each task.

Our results suggest that supportive contents like tutorials should give detailed information about the capabilities of the application. Also, these interfaces should be designed

by considering the mental model of non-programmer adults. Therefore, designers should use familiar metaphors for describing programming concepts as well as using graphical elements that can indicate the functional relationship of code blocks. Finally, our findings revealed the need for building an easy-to-understand spatial relationship between the digital interface and the robotic toy.

Acknowledgments. Thanks to İdil Bostan and Tülin Bozkurt Hazar for proofreading.

References

1. Klassner, F., Anderson, S.D.: LEGO mindstorms: not just for K-12 anymore. *IEEE Robot. Autom. Mag.* **10**, 12–18 (2003)
2. Ramírez-benavides, K., López, G., Guerrero, L.A.: A mobile application that allows children in the early childhood to program robots. *Mob. Inf. Syst.* **2015** (2016). Article ID: 1714350
3. Diprose, J.P., MacDonald, B.A., Hosking, J.G.: Ruru: a spatial and interactive visual programming language for novice robot programming. In: *Proceedings of the 2011 IEEE Symposium on Visual Languages and Human-Centric Computing VL/HCC 2011*, pp. 25–32 (2011)
4. Weintrop, D., Wilensky, U.: To block or not to block? That is the question. *J. Educ. Res.* **95**(4), 196–202 (2015)
5. Dill, K., Freeman, B., Frazier, S.: Mars game: creating and evaluating an engaging educational game. In: *Interservice/Industry Training, Simulation, and Education Conference (IITSEC)*, pp. 1–14 (2015)
6. Pane, J.F.: A programming system for children that is designed for usability. Dissertation, Carnegie Mellon University (2002)
7. Elkin, M., Sullivan, A., Bers, M.U.: Programming with the KIBO robotics kit in preschool classrooms. *Comput. Sch.* **33**, 169–186 (2016)
8. Zelazo, P., Carter, A., Reznick, J.S., Frye, D.: Early development of executive function. *Rev. Gen. Psychol.* **1**(2), 198–226 (1997)
9. Lee, K.T.H., Sullivan, A., Bers, M.U.: Collaboration by design: using robotics to foster social interaction in kindergarten. *Comput. Sch.* **30**, 271–281 (2013)
10. Begel, A.: LogoBlocks: a graphical programming language for interacting with the world. Electrical Engineering and Computer Science Department, MIT, Boston, MA (1996)
11. Bau, D., Bau, D.A., Pickens, C.S., Dawson, M.: Pencil code: block code for a text world. In: *IDC*, pp. 445–448 (2015)
12. Resnick, M., Maloney, J., Monroy-Hernández, A., Rusk, N., Eastmond, E., Brennan, K., Millner, A., Rosenbaum, E., Silver, J., Silverman, B., Kafai, Y.: Scratch: programming for all. *Commun. ACM.* **52**, 60–67 (2009)
13. Cheung, J.C.Y., Ngai, G., Chan, S.C.F., Lau, W.W.Y.: Filling the gap in programming instruction: a text-enhanced graphical programming environment for junior high students. *SIGCSE Bull.* **41**, 276–280 (2009)
14. García-peñalvo, F.J., Rees, A.M., Hughes, J., Vermeersch, J.: A survey of resources for introducing coding into schools. In: *Proceedings of the Fourth International Conference on Technological Ecosystems for Enhancing Multiculturality* (2016)
15. Kim, S.H., Jeon, J.W.: Programming LEGO mindstorms NXT with visual programming Seung. In: *International Conference on Control, Automation and Systems*, Seoul, Korea, pp. 2468–2472 (2007)

16. Ng, G., Chow, M., de Lima Salgado, A.: Toys and mobile applications: current trends and related privacy issues. In: Hung, P.C.K. (ed.) *Mobile Services for Toy Computing*, pp. 51–76. Springer (2015)
17. Ketola, P., Roto, V.: Exploring user experience measurement needs. In: *Proceedings of the International Workshop on Meaningful Measures: Valid Useful User Experience Measurement (VUUM)*, Reykjavik, Iceland, pp. 23–26 (2008)
18. Shackel, B., Richardson, S.J.: *Usability—Context, Framework, Definition, Design and Evaluation*. Cambridge University Press, New York, NY (1991)
19. Nielsen, J.: *Usability Engineering*. Elsevier, Englewood Cliffs (1993)
20. Jordan, P.W.: *An Introduction to Usability*. CRC Press, USA (1998)
21. Hung, J.: *Usability and learnability improvements for the TaleBlazer game editor*. Dissertation, Institute of Technology, Massachusetts (2015)
22. Nielsen, J., Landauer, T.K.: A mathematical model of the finding of usability problems. In: *ACM INTERCHI 1993*, pp. 206–213 (1993)
23. Nielsen, J.: How many test users in a usability study? (2012). <https://www.nngroup.com/articles/how-many-test-users/>. Accessed 8 Feb 2017
24. Fonteyn, M.E., Kuipers, B., Grobe, S.J.: A description of think aloud method and protocol analysis. *Qual. Health Res.* **3**, 430–441 (1993)
25. Brooke, J.: SUS - a quick and dirty usability scale. *Usability Eval. Ind.* **189**, 4–7 (1996)